

progetto asd

PIETRO FUBINI

May 2026

1 Come gestire i dati

come deve essere il grafo finale:

1. lista hasciata dei nodi
2. ogni nodo e una n-upla (nome nodo, hascing function, lista hasciata di adiacenza)
3. per altre funzioni si è visto che è conveniente mettere anche il numero di nodi e il peso massimo assunto da un arco

ogni nodo (formato dal nome nodo e la funzione di hascing universale) ha la lista di adiacenza hasciata con hascing universale
la lista di adiacenze è formata da coppie, con il vicino e il peso

i nodi sono a loro volta hasciati in una tabella per cercarli bene

1.1 input file

- input: prende il file **all-paths** con un cin
- grafo non orientato $G=(n,e)$, con edge pesati
- funzionalità:
 1. crea tutti i vari archi pian piano che li legge usando la funzione aggiungi arco,
 2. chiude il file
- si è visto a posteriori che è molto conveniente usare molte funzioni secondarie tipo `increase weight`, perché tutto quanto è hasciato, o altre funzioni tipo "rendi leggibile il file che si occupa di parsare la linea"

1.2 funzioni utili

1.2.1 albero puntato

- **input** : nodo partenza e arrivo, cap sulla lunghezza
- **algoritmo** : prende un nodo e scorre tutta la sua lista di adiacenza, aggiungendo tutti i nodi con meno della distanza max, ogni nodo è quindi aggiunto, i vari nodi dell'albero sono fatti in questa maniera:

1. nome nodo
2. nodo babbo
3. insieme hasciato di nodi già percorsi

continua questo processo se trova il nodo di arrivo si ferma, la lista hasciata dei nodi già percorsi serve perché un percorso non può così passare 2 volte dallo stesso nodo, infatti negli step prima di aggiungere il nodo il programma checkerà che il nodo che sta per aggiungere non sia già stato visitato, si ferma in 2 casi o il nodo che sta cercando di espandere è il nodo 2 o non ha più nodi su cui espandere (questo caso succede nella situazione dove ha già visitato tutti i nodi vicini per esempio), in questo caso chiama la funzione potatura su questo nodo per eliminare questo cammino non fruttuoso

- **output** : un albero con tutti i percorsi, ovviamente dato che cecca di non rivisitare nodi ogni ramo fogliato è un percorso diverso, si potrebbe fare anche guardando di non ribeccare rami ma, l'algoritmo cambierebbe aggiungerebbe solo complessità computazionale
- **costo computazionale**: la creazione dei figli che quindi include:
 1. aggiungere un figlio per ogni vicino controllando la lunghezza $O(n)$ dato che al massimo ogni nodo ha n vicini dove n è il numero di vertici, ma questa è una stima molto dall'alto
 2. ogni path è al più lungo n e ci sono al più $n!$ percorsi (ogni percorso può essere visto come una permutazione dei nodi, ma questo è sufficiente contare solo quelle lunghe n perché prima di tutto sono chiaramente lunghe al più n e perché ogni segmento iniziale che è un cammino valido non può essere espanso quindi non c'è il rischio di associare a 2 cammini la stessa permutazione), quindi il costo computazionale è $O((n+1)!)$

1.2.2 potatura

- **input** input: nodo senza figli,
- **procedimento**: cancella se stesso e passa la funzione al nodo genitore, se questo ora non ha figli runna di nuovo potatura, in questo modo taglia completamente un ramo

- **output:**albero senza il ramo del nodo
- **costo computazionale:** ci mette $O(1)$ per ogni nodo che cancella quindi nel caso peggiore l'algoritmo ci mette $O(\text{depth})$

1.2.3 lista adiacenza

prende in input un nodo e ritorna un vettore di coppie con la sua lista di adiacenza con i pesi

1.2.4 aggiungi nodo

aggiunge il nodo, randomizzando una hash universale, prende in input un int(i vari nodi sono degli int) e per il grafo de-referenziato

1.2.5 aggiungi arco

input: nodo 1 e nodo 2

algoritmo: va in nodo uno e cerca nella lista di adiacenza con la funzione di hasching il nodo 2 se non ce lo aggiunge con il peso 1 altrimenti aumenta di 1 il peso fa lo al contrario, rehasha se necessario
in teoria è $O(1)$

output: è aggiunto in entrambe delle liste di adiacenza in più uno su quell'arco

costo computazionale: è chiaramente $O(1)$, dato che quasi certamente non dovrà ri-hashare la lista dei vicini ed anche in quel caso ce il numero di vicini è ragionevolmente abbastanza basso, in media quindi il costo sarà $O(1)$

1.2.6 getweightto

- **input:** nodo di arrivo (è una funzione sotto un nodo, quindi non ha bisogno del nodo di partenza)
- **processo:** cerca il nodo e prende il peso
- **output:** il peso del nodo
- **costo:** cercare il nodo $O(1)$ e prendere il peso idem

2 da .all-paths a struttura di grafo

prende il testo linea linea e sposta il grafo. ogni passaggio prende 2 rami, checka se i vertici esistono già e si aggiunge o somma uno alla lunghezza del ramo altrimenti aggiunge i vertici e aggiunge 1 alla lunghezza del ramo. chiaramente il ramo non esistente conta come 0 anche se non lo creo per questioni di memoria
è importante ridia anche la lunghezza massima di un arco

3 funzioni utili

3.1 conta foglie

- **input:** testa di un albero
- **processo:** ricorsivo:
 - passo base: il nodo non ha figli ritorna 1;
 - passo induttivo il nodo ha figli, ritorna la somma del risultato dei figli
- **output:** numero di foglie
- **costo computazionale:** visita ogni foglia 1 volta e su ogni foglia ci la funzione ci mette $O(1)$ quindi il costo è $O(n)$ dove n è il numero di nodi dell'albero

3.2 bargraph

- **input:** un vettore, un titolo (stringa), il massimo valore
- **processo:** stampa la stringa titolo e poi una serie di barrette in questo modo:

```
titolo
-----
peso 1 |##### {31}
peso 2 |##### {17}
peso 3 |##### {37}
peso 4 |##### {29}
```

Poi in inizia a plottare in questo modo :

1. quando la size del vettore che ti dice quanto il label (peso 1 |) quanto spazio ci deve essere tra n e | ovviamente contando il numero di cifre
2. stampa "peso n |" che prenderà sempre un numero di caratteri sulla linea fissato e il numero di caratteri che ci sottrarre altre 8 caratteri che servono a scrivere il valore di quell'arco mi aspetto che il peso sia al massimo peso 1 peso 1 | un milione (100000), questo numero che rimane è sempre costante
guarda quando spazio gli ci vuole per scrivere "peso n |" = l che è lo stesso per tutti gli n e ci sottrae 9 caratteri che servono a scrivere il numero di archi con quel peso (con uno spazio), per esempio " (100000)" in generale mi aspetto che nessun arco sia percorso un milione di volte.

so che sul terminale posso stampare al massimo k caratteri e che ne ho già usate $l+9$, $k - (l+9) = x$. Quindi per capire quanti cancelletti stampare uso questa formula $\lfloor x \cdot \frac{v[p]}{\max \text{ percorrenza}} \rfloor$ che dà il numero di cancelletti da stampare, e così via

- **output:** un grafico
- **costo:** ogni stampa è $O(1)$ quindi stampare tutto costa $O(v.size)$

4 minimo percorso tra 2

- **tipologia algoritmo:** Greedy, programmazione dinamica.
- **input** grafo pesato, nodi da collegare
- **algoritmo:** Prende il nodo e (controlla se ce l'ultimo nodo) lascia i suoi vicini con la funzione di hashing che dà la distanza dal vertice di partenza a quello valutato, e poi lascia universalmente in ogni set di lunghezza, e poi fa la ricorsione (sul primo della lista relativa al percorso più leggero), passandosi ogni volta che fa il numero più alto preso.
ce infine un'ultima lista lasciata dei nodi già visitati che viene tenuta per evitare di riesplorare più volte il lo stesso nodo, tanto o per lo stesso ragionamento di Dijkstra non ce un percorso più vicino che ci arriva
l'algoritmo si conclude quando dopo un'espansione di un nodo il nodo cercato si trova nella lunghezza in quel momento minore, se ce interrompe l'algoritmo facendo il max tra il max per ora e la lunghezza del ramo cercato

i casi doppi: per evitare questi casi l'algoritmo, prima di aggiungere un nodo nella lista di quelli da controllare guarda, innanzitutto se è già stato controllato, se si passa al prossimo, se non è ancora stato controllato guarda se è già in lista per lunghezze minori, e per sicurezza anche in quelle maggiori, per evitare di riempire la struttura di spazzatura

- **output** distanza tra i due
- **costo computazionale:** passa da ogni nodo al massimo 1 volta (visto che prima di aprire 1 nodo conta se è già stato aperto). Quando è nel nodo:
 1. guarda tutti i vicini, costo $O(n)$
 2. in ogni vicino controlla se ce nella lista dei già visitati costo $O(1)$, se no, allora controlla se e dove sta nella lista delle lunghezze, è ragionevole supporre che il numero di lunghezze sia basso, e di conseguenza che questo costo visto che ad ogni lunghezza pesa $O(1)$ sia comunque $O(1)$, espande tutti i vicini e li mette nella lista e questo costa $n \cdot O(1)$

3. Dopo aver espanso un nodo controlla che ci sia nella lista lasciata della lunghezza più bassa il nodo cercato, anche questo è $O(1)$

da questa analisi sembra essere che il costo computazionale sia $O(n^2)$ ma questo è il caso peggiore e con ampio margine anche perché se il grafo è molto connesso, certo l'espansione costa tanto ma al contempo probabilmente trovo in fretta un vicino ottimale al mio elemento, se invece il grafo è poco connesso devo controllare meno nodi ma allo stesso tempo ho molti meno elementi quando espando.

5 numero di cammini min max

- **input:** grafo, nodo 1, nodo 2, costo min max (implementato in modo tale che se viene dato come -1 è considerato come non lo so)
- **algoritmo:** chiaramente se non sa il costo minimo tra 2 nodi lo calcola poi crea l'albero con la funzione 1.2.1 e poi conta le foglie con la funzione 3.1
- **output:** l'albero e il numero di foglie
- **costo computazionale:** sembra essere molto grande, perché si divide nei seguenti step additivi:
 1. eventuale costo di contare i cammini $O(n^2)$
 2. costruzione albero $O((n+1)!)$
 3. contare le foglie che costa come il numero di nodi quindi dato che ci sono al più $n!$ cammini lunghi n con una stima molto molto larga il costo computazionale è $O((n+1)!)$

che si sommano quindi a $O((n+1)!)$

6 info graph

- **input:** grafo universale
- **processo:**
 1. numero nodi: list.size
 2. numero archi: scorre i nodi della lista, e somma il numero di archi e divide per 2 alla fine, ricordandosi quale sia il nodo più connesso. Mentre scorre crea un vettore (v), e su ogni posto ti dice quanti ogni volta che prende un arco di peso p esegue $v[p] += \frac{1}{2}$; a fine dell'esecuzione guarda il vettore per trovare l'arco più frequente. **C'è da stare molto attenti con il vettore ogni volta che aggiunge un nuovo valore al peso deve controllare prima di tutto che il peso esista e se non esiste, lo allunga usando resize**

3. nodo più connesso: stampa il nodo più connesso
4. grafico delle frequenze: usa la funzione grafico e stampa quindi tutto

- **output:**

1. numero nodi
2. numero archi
3. nodo più connesso
4. grafico delle frequenze

- **costo computazionale:** per il punto 1 il costo immagino sia $O(1)$, mentre il punto 4 e 3 usano lo i calcoli fatti nel punto 2 (non ha davvero senso guardare quanto costa il tempo di printing perché è rallentato dal terminale e non dal codice, anche guardando con i tempi del terminale sarebbe quasi tempo di stampare un carattere·numero caratteri su una riga·($v.size + 2$)).

quindi il costo del punto 2 calcolo guardando le operazioni che si "nestano" tra di loro

1. scorro ogni nodo costo: $O(n)$
2. in ogni nodo, conto gli archi $O(1)$ (guardo la size del vettore di adiacenza), ora però li devo scorrere per guardare il loro peso eventualmente allungando il vettore, ma sono tutte operazioni $O(1)$ quindi il costo totale è $O(n)$

quindi il costo totale è $O(n^2)$

7 Main e UI

quando avvii il programma ti chiede il nome del file ad immettere, e una volta creato il grafo ti stampa la lista di tutti i nodi, infine chiede quale delle funzioni eseguire, da lì agisce di conseguenza sempre stampando il tempo per runnare l'algoritmo, quindi ti rifà le domande (tra le domande ce sempre se vuoi cambiare il file da loadare o chiudere il programma etc etc) **evidenze sperimentali hanno mmostrtrato che è utile mettere un cap sul numero che vuoi leggere**

8 upgrade necessari post test-codice

8.1 output

si è visto subito necessario visto che il terminale dopo relativamente poco taglia l'output che è molto più comodo (necessario) stampare su file

8.2 checkpoint

potrebbe essere carino stampare dei checkpoint che dicono quando si è arrivati ad un certo punto dell'algoritmo

8.3 componenti connesse

sarebbe carino un algoritmo che ti da le componenti connesse del grafo:

- **input:** grafo
- **processo:** prende un nodo e continua ed espandere la lista di adiacenza fino a creare dei set, quindi crea la prima componente connessa, cerca poi un nodo nel complementare rispande e così via (magari in un set in modo tale da non mettere doppioni, o in una lista lasciata per rendere comunque l'inserzione $O(1)$), prende dal database di nomi chiamato nomi.txt un nome a caso e da un nome ad ogni componente connessa
- **output:** crea n vettori (dove ognuna è una componente connessa) quindi viene un vettore di vettori, stampa poi le componenti connesse (tipo componente connessa 1), e ti chiede se ne vuoi stampare una (ovviamente poi fa tutte le altre domande ragionevoli [altre componenti connesse o trovare le domande])
- **costo computazionale:** una volta trovato un'espansione visto che l'aggiunzione costa $O(1)$ costa in totale $O(n)$, invece il costo di cercare il nodo supponendo che il grafo sia abbastanza connesso (assunzione molto ragionevole) costa al più n controlli ogni volta che metto e cerco una nuova componente connessa, e vista la supposizione il numero di componenti connesse è molto basso il costo di trovare un nuovo elemento sconnesso, è $O(n)$, il costo totale è quindi $O(n)$